

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Pamięć

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 10

Studia dzienne

Informatyka

Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

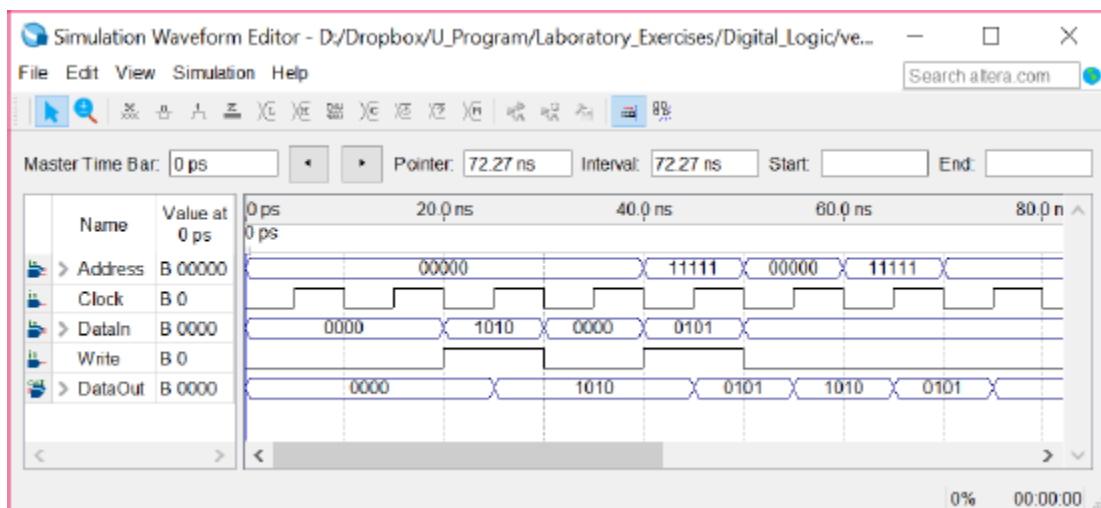
Data: 19.05.2022

Zadanie 1

Utwórz projekt ram32x4 jednoportowej pamięci RAM o objętości 32 4-bitowych słów przy użyciu funkcji bibliotecznej (IP Catalog), która jest zdefiniowana przez następujący moduł:

```
module (ram 32x4(  
    input[4:0] address,  
    input clk,  
    input[3:0] data,  
    input wren,  
    output[3:0] q);
```

Utwórz nowy projekt my_ram32x4 wysokiego poziomu, w którym utworzona będzie instancja modułu ram32x4. Symuluj obwód (rys. 1), upewnij się, że dane można zapisać w pamięci i odczytać z pamięci.



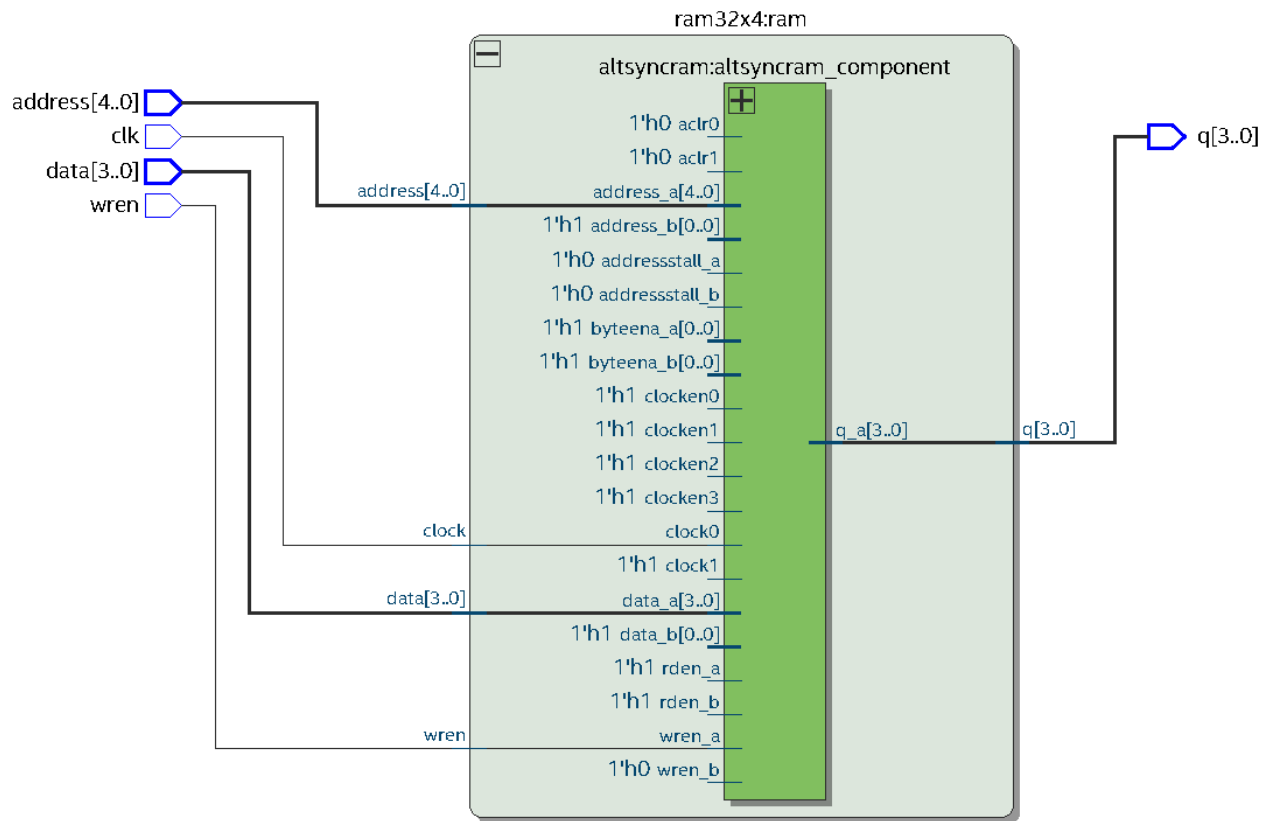
Rys. 1. Przykład rezultatów symulacji

Kod w Verilog

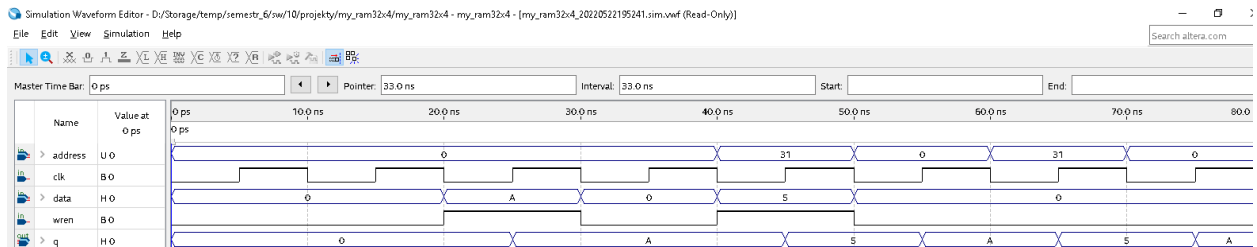
Moduł wysokiego poziomu korzystający z modułu ram32x4 wygenerowanego za pomocą Tools > IP Catalog > Basic Functions > On Chip Memory > RAM: 1-PORT.

```
module my_ram32x4(  
    input [4:0] address,  
    input clk,  
    input [3:0] data,  
    input wren,  
    output [3:0] q);  
  
    ram32x4 ram(address, clk, data, wren, q);  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 2

Zaimplementuj projekt my_ram32x4 na płycie z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
address	SW8-4, HEX5-4
clk	KEY0
data	SW3-0, HEX2
wren	SW9
q	HEX0

Sprawdź układ na płycie.

Kod w Verilog

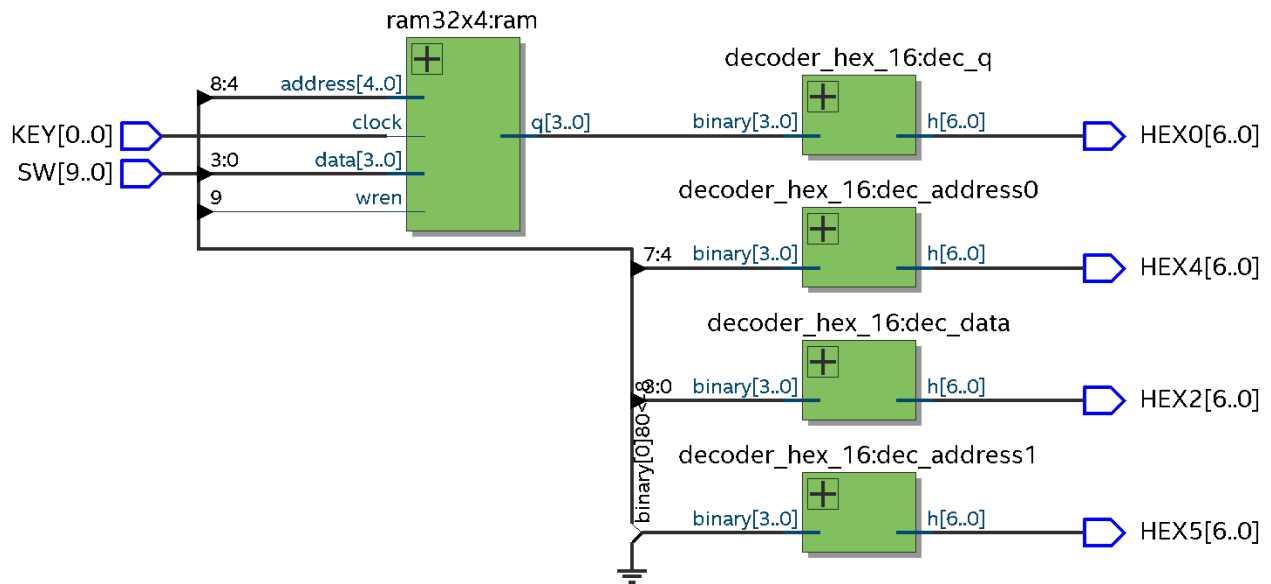
Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module my_ram32x4_on_board(  
    input [9:0] SW,  
    input [0:0] KEY,  
    output [6:0] HEX5, HEX4, HEX2, HEX0);  
  
    wire [3:0] q /* synthesis keep */;  
    ram32x4 ram(SW[8:4], KEY[0], SW[3:0], SW[9], q);  
    decoder_hex_16 dec_address1({3'b000, SW[8]}, HEX5);  
    decoder_hex_16 dec_address0(SW[7:4], HEX4);  
    decoder_hex_16 dec_data(SW[3:0], HEX2);  
    decoder_hex_16 dec_q(q, HEX0);  
  
endmodule
```

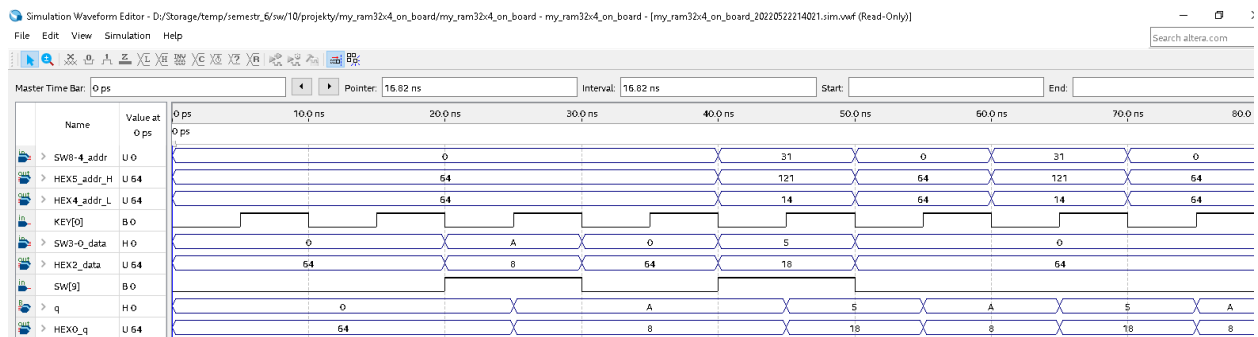
Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_16(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            4'd10: h = 7'b0001000; // 8; A  
            4'd11: h = 7'b0000011; // 3; b  
            4'd12: h = 7'b0100111; // 39; c  
            4'd13: h = 7'b0100001; // 33; d  
            4'd14: h = 7'b0000110; // 6; E  
            4'd15: h = 7'b0001110; // 14; F  
        endcase  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 3

Wykonaj poprzedni projekt, deklarując pamięć w kodzie:

reg[3:0] memory_array[31:0];

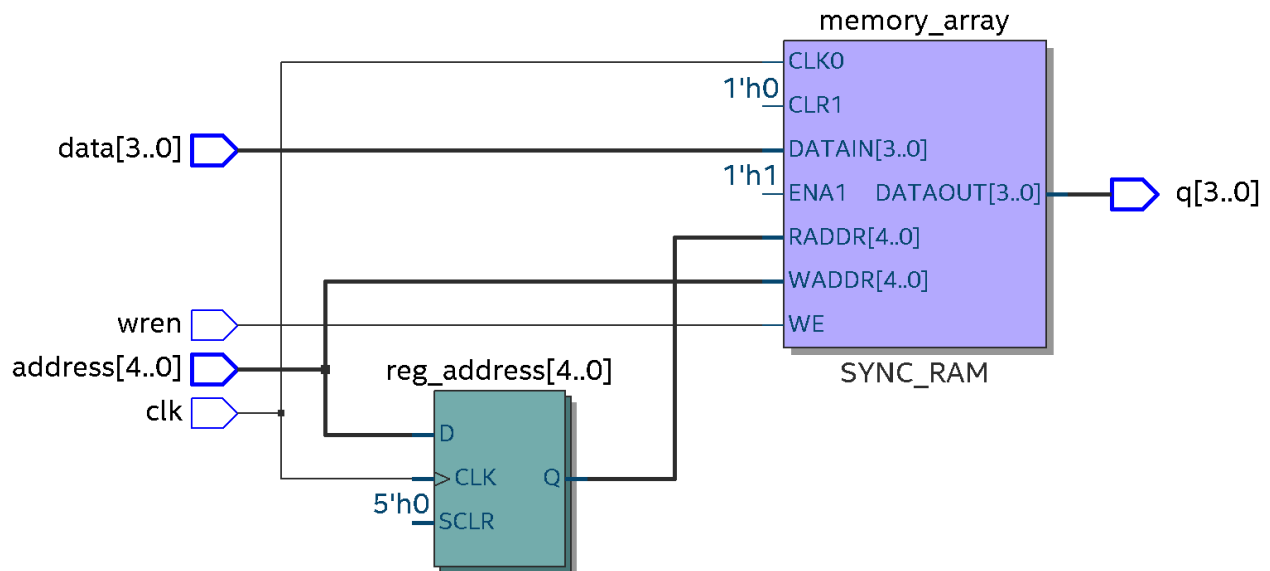
Wykonaj symulację i implementację projektu na płycie.

Kod w Verilog

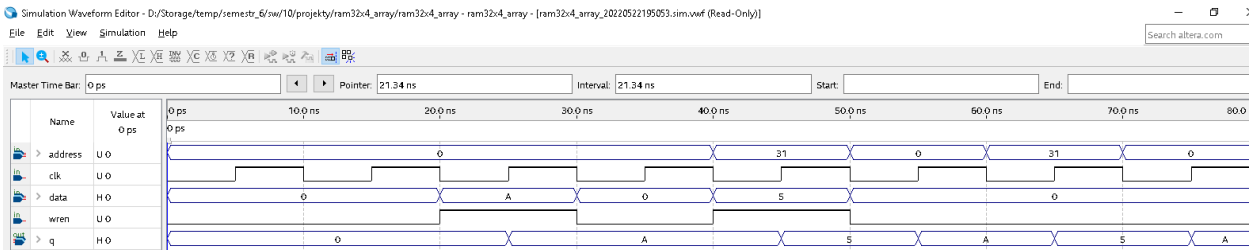
Moduł z pamięcią w kodzie

```
module ram32x4_array(  
    input [4:0] address,  
    input clk,  
    input [3:0] data,  
    input wren,  
    output [3:0] q);  
  
    reg [3:0] memory_array[31:0];  
    reg [4:0] reg_address;  
    reg [3:0] reg_data;  
    reg reg_wren;  
    always @ (posedge clk) begin  
        reg_address <= address;  
        reg_data <= data;  
        reg_wren <= wren;  
    end  
    always @ (*)  
        if (reg_wren) memory_array[reg_address] = reg_data;  
    assign q = memory_array[reg_address];  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 4

Korzystając z funkcji biblioteki IP, utwórz projekt ram32x4_2_ports dla pamięci dwuportowej. Połącz plik inicjujący MIF z projektem (patrz rys. 2).

```
DEPTH = 32;  
WIDTH = 4;  
ADDRESS_RADIX = HEX;  
DATA_RADIX = BIN;  
CONTENT  
BEGIN  
  
0 : 0000;  
1 : 0001;  
2 : 0010;  
3 : 0011;  
... (some lines not shown)  
1E : 1110;  
1F : 1111;  
  
END;
```

Rys. 2. Przykład pliku inicjującego pamięć (**MIF**)

Zaimplementuj projekt na płycie w taki sposób, aby można było zobaczyć zawartość każdego słowa pamięci przez około 1 sekundę. Przypisanie pinów:

Wyprowadzenia	Piny
q	HEX0
data	HEX1
read_addr	HEX3-2
write_addr	HEX5-4
clk	CLOCK_50

Wskazówka: Użyj 5-bitowego licznika do określenia adresu odczytu z pamięci, użyj projektu del_1_sec do wygenerowania impulsu o długości 1 s.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module iterating_ram_on_board(  
    input CLOCK_50,  
    input [9:0] SW,  
    input [0:0] KEY,  
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5);  
  
    wire [4:0] read_addr;  
    wire [3:0] q;  
    iterating_ram ir(CLOCK_50, SW[3:0], SW[8:4], SW[9], KEY[0], read_addr, q);  
    decoder_hex_16 dec_q(q, HEX0);  
    decoder_hex_16 dec_data(SW[3:0], HEX1);  
    decoder_hex_16 dec_read_addr1({3'b000, read_addr[4]}, HEX3);  
    decoder_hex_16 dec_read_addr0(read_addr[3:0], HEX2);  
    decoder_hex_16 dec_write_addr1({3'b000, SW[8]}, HEX5);  
    decoder_hex_16 dec_write_addr0(SW[7:4], HEX4);  
  
endmodule
```

Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_16(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            4'd10: h = 7'b0001000; // 8; A  
            4'd11: h = 7'b0000011; // 3; b  
            4'd12: h = 7'b0100111; // 39; c  
            4'd13: h = 7'b0100001; // 33; d  
            4'd14: h = 7'b0000110; // 6; E  
            4'd15: h = 7'b0001110; // 14; F  
        endcase  
  
endmodule
```

Moduł pozwalający zobaczyć zawartość słów z kolejnych adresów pamięci przez czas zależny od modułu opóźnienia delay

```
module iterating_ram(
    input clk,
    input [3:0] data,
    input [4:0] write_addr,
    input write_enable, reset_counter,
    output [4:0] read_addr,
    output [3:0] q);

    wire clk_1_sec;
    delay_1_sec delay(clk, clk_1_sec);
    counter_N_bits #(5) address_counter(clk_1_sec, reset_counter, read_addr);
    ram32x4_2_ports ram(clk, data, read_addr, write_addr, write_enable, q);

endmodule
```

Moduł N-bitowego licznika

```
module counter_N_bits
    #(parameter N = 4)(
        input clock, areset,
        output reg [N-1:0] state);

    initial state = {N{1'b0}};
    always @ (posedge clock, negedge areset)
        if (~areset) state <= {N{1'b0}};
        else state <= state + 1'b1;

endmodule
```

Moduł opóźnienia generujący sygnał zegara slow_clock z interwałem 1 sekundy. Zegar CLOCK_50 w płycie DE1-SoC ma częstotliwość 50 MHz, więc w module opóźnienia używamy licznika modulo 50 000 000.

```
module delay_1_sec(
    input clock,
    output slow_clock); // zbocze 0 -> 1 w momencie wyzerowania (przekręcenia) licznika

    wire clock_rollover;
    counter_modulo_k #(50_000_000) counter(clock, clock_rollover);
    assign slow_clock = ~clock_rollover;

endmodule
```

Moduł licznika modulo k

```
module counter_modulo_k
    #(parameter k = 20)( // okres licznika
        input clk,
        output rollover);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

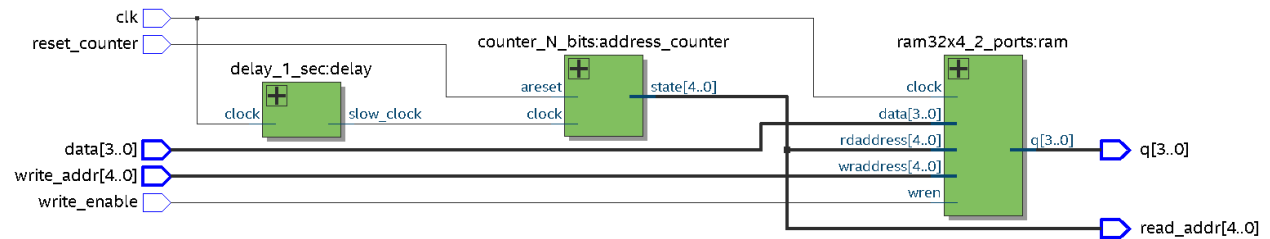
    localparam N = clogb2(k-1); // długość licznika w bitach

    reg [N-1:0] Q = {N{1'b0}};
    always @ (posedge clk)
        if (Q == k-1) Q <= {N{1'b0}};
        else Q <= Q + 1'b1;

    assign rollover = (Q == k-1);

endmodule
```

Wyniki syntezy



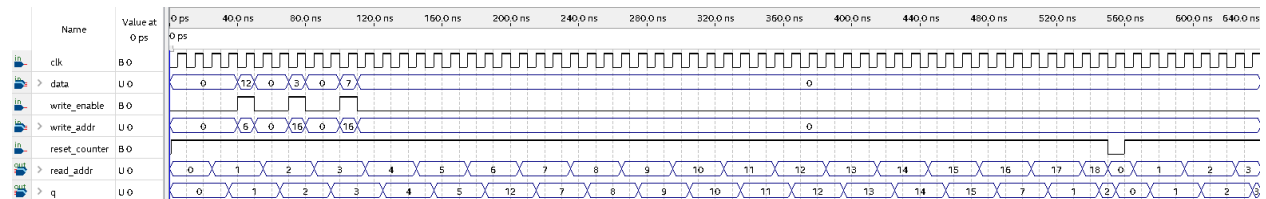
Wyniki symulacji funkcjonalnej

Przyjmujemy następującą zawartość pliku inicjującego pamięć (MIF):

```
WIDTH=4;
DEPTH=32;
ADDRESS_RADIX=HEX;
DATA_RADIX=BIN;
CONTENT BEGIN
    00 : 0000;
    01 : 0001;
    02 : 0010;
    03 : 0011;
    04 : 0100;
    05 : 0101;
    06 : 0110;
    07 : 0111;
    08 : 1000;
    09 : 1001;
    0A : 1010;
    0B : 1011;
    0C : 1100;
    0D : 1101;
    0E : 1110;
    0F : 1111;
    10 : 0000;
    11 : 0001;
    12 : 0010;
    13 : 0011;
    14 : 0100;
    15 : 0101;
    16 : 0110;
    17 : 0111;
    18 : 1000;
    19 : 1001;
    1A : 1010;
    1B : 1011;
    1C : 1100;
    1D : 1101;
    1E : 1110;
    1F : 1111;
END;
```

Aby w symulacji było widać działanie układu, parametryzujemy moduł opóźnienia delay_1_sec jako licznik modulo 3. Wówczas wartość licznika adresu odczytu z pamięci (read_addr) zwiększa się o 1 co 3 zbocza narastające zegara clk.

Pamięć na wszystkich adresach od 0 do 31 jest inicjowana zgodnie z plikiem MIF. Można na adresie pamięci write_addr wpisać słowo data za pomocą narastającego zbocza zegara clk, kiedy write_enable = 1.



Wnioski

Wszystkie zadania zostały zrealizowane.